ADAM

Using gradient descent brings a problem of stability based on the function: the steeper is my function, the longer will be the time to reach the minimum, and viceversa. Empirically it is tested that the loss function is explorable properly with ADAM (theoretical proof is poor though). ADAM is based on making fixed steps on the function:

$$m_{t+1} \leftarrow \frac{\partial L(\varphi_t)}{\partial \varphi} \qquad v_{t+1} \leftarrow \left(\frac{\partial L(\varphi_t)}{\partial \varphi}\right)^2$$

Let's fix the steps according to this formula:

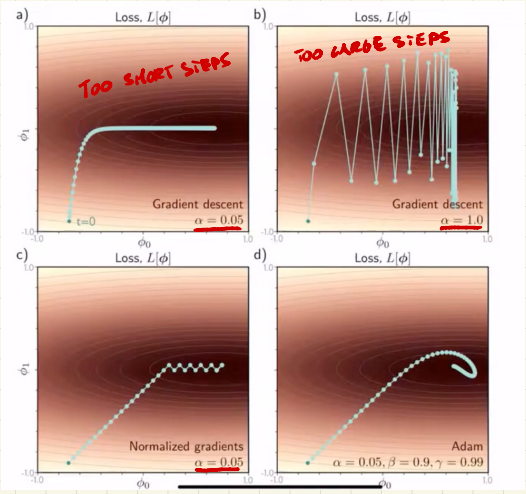
NORMALIZED GRADIENT

$$\varphi_{t+1} \leftarrow \varphi_t - \alpha \frac{m_{t+1}}{\sqrt{v_{t+1}} + \varepsilon}$$

NORM = 1

compared to gradient descent, I am just isolating the sign of the gradient descent (that's why we use $\sqrt{v_{t+1}}$). So I am just using gradient descent direction and, by learning rate, how further my step should be alone.

$$\varphi_{t+1} \leftarrow \varphi_t - \alpha \frac{\partial L(\varphi_t)}{\partial \varphi}$$



LINEAR REGRESSION EXAMPLE (2 PARAMETERS: φ_0 - φ_1)

Now, by mixing MOMENTUM and NORMALIZED GRADIENT:

ADAM

$$m_{t+1} \longleftarrow \beta m_t + (1-\beta)\frac{\partial \mathcal{L}(\varphi_t)}{\partial \varphi}, \quad \beta \in [0,1)$$

$$v_{t+1} \longleftarrow \gamma v_t + (1-\gamma)\left(\frac{\partial \mathcal{L}(\varphi_t)}{\partial \varphi}\right)^2, \quad \gamma \in [0,1)$$

$$\varphi_{t+1} \longleftarrow \varphi_t - \alpha\left(\frac{m_{t+1}}{\sqrt{v_{t+1}+\epsilon}}\right) \rightarrow \text{NORM} \neq 1$$

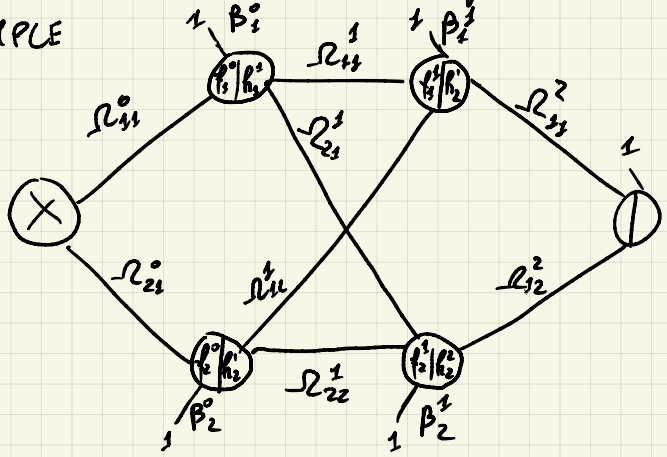
The hyperparameters of this algorithm are 3:

- learning rate (α)
- β, γ

ADAM is the general standard algorithm in order to fit a neural network to some data.

N.B.: Neural network fitting as they are seen here rely on loss function DIFFERENTIABILITY (without it, we can't rely to GRADIENT DESCENT, which leads to the direction where the loss function is getting reduced)

EXAMPLE



f: PREACTIVATION FUNCTION

h: ACTIVATION FUNCTION

$$f^0 = \begin{bmatrix} f_1^0 \\ f_2^0 \end{bmatrix} = \begin{bmatrix} \beta_1^0 + \Omega_{11}^0 x \\ \beta_2^0 + \Omega_{21}^0 x \end{bmatrix} = \begin{bmatrix} \beta_1^0 \\ \beta_2^0 \end{bmatrix} + \begin{bmatrix} \Omega_{11}^0 \\ \Omega_{21}^0 \end{bmatrix} x$$

β^0 $\quad$ Ω^0

$$h^1 = \begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = \begin{bmatrix} ReLU(f_1^0) \\ ReLU(f_2^0) \end{bmatrix} = ReLU\left(\begin{bmatrix} f_1^0 \\ f_2^0 \end{bmatrix}\right)$$

$$f^1 = \begin{bmatrix} f_1^1 \\ f_2^1 \end{bmatrix} = \begin{bmatrix} \beta_1^1 + \Omega_{11}^1 h_1^1 + \Omega_{12}^1 h_2^1 \\ \beta_2^1 + \Omega_{21}^1 h_1^1 + \Omega_{22}^1 h_2^1 \end{bmatrix} = \begin{bmatrix} \beta_1^1 \\ \beta_2^1 \end{bmatrix} + \begin{bmatrix} \Omega_{11}^1 & \Omega_{12}^1 \\ \Omega_{21}^1 & \Omega_{22}^1 \end{bmatrix} \begin{bmatrix} h_1^1 \\ h_2^1 \end{bmatrix} = f^1 \Omega^1 h^1$$

$$h^2 = \begin{bmatrix} h_1^2 \\ h_2^2 \end{bmatrix} = ReLU\left(\begin{bmatrix} f_1^1 \\ f_2^1 \end{bmatrix}\right) = ReLU(f^1)$$

$$f^2 = \beta_0^2 + \Omega_{11}^2 h_1^2 + \Omega_{12}^2 h_2^2 = \beta^2 + \begin{bmatrix} \Omega_{11}^2 & \Omega_{12}^2 \end{bmatrix} \begin{bmatrix} h_1^2 \\ h_2^2 \end{bmatrix} = \beta^2 \Omega^2 h^2$$

How will the loss function be?

$S = \{x, y\}$

$L(\Omega, \beta) = \frac{1}{2}(f^2 - y^2)^2$

$L(\Omega, \beta)$ INDICATES THAT L DEPENDS ON ALL ΩS AND βS, WHICH ARE THE PARAMETERS THAT NEED TO BE FIND (x, y, h ARE GIVEN)

Let's compute the gradient:

$$L = \frac{1}{2}(f^2 - y^2)^2 =$$

$$f^2 = \beta^2 + \Omega^2 h^2 = \beta^2 + \Omega^2 ReLU(f^1) = \beta^2 + \Omega^2 ReLU(\beta^1 + \Omega^1 h^1) = \dots =$$

$$= \dots$$ KEEP EXPANDING

f is a composition of functions, so, in order to get the derivative, we need to compute the derivative of a composition of functions:

$$\frac{\partial(f(g(x)))}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

To get it, we will use an algorithm that computes the gradient, called BACKPROPAGATION (BACKPROP).

$$S = \{(x, y)\}$$

$$L = \frac{1}{2}(f^2 - y^2)^2$$

ALWAYS NEEDED

$$\frac{\partial L}{\partial \Omega^1_{11}} = \boxed{\frac{\partial L}{\partial f^2}} \cdot \frac{\partial f^2}{\partial \Omega^1_{11}} = \boxed{\frac{1}{2} \cdot 2(f^2 - y)} \cdot \frac{\partial f^2}{\partial \Omega^1_{11}}$$

↖ in f^2 → DERIVATIVE OF COMPOSITION OF FUNCTIONS

In every partial derivative, $\frac{\partial L}{\partial f^2}$ will always occur.

$$\frac{\partial f^2}{\partial \Omega^1_{11}} = \frac{\partial(\beta^2 + \Omega^2_{11} h^2_1 + \Omega^2_{12} h^2_2)}{\partial \Omega^1_{11}} = \frac{\partial(\Omega^2_{11} h^2_1)}{\partial \Omega^1_{11}} = \Omega^2_{11} \frac{\partial h^2_1}{\partial \Omega^1_{11}} \Rightarrow$$

$$\frac{\partial h_1^2}{\partial R_{11}^1} = \frac{\partial ReLU(f_1^1)}{\partial R_{11}^1} = \frac{\partial ReLU(f_1^1)}{\partial f_1^1} \cdot \frac{\partial f_1^1}{\partial R_{11}^1} = step(f_1^1) \cdot \frac{\partial f_1^1}{\partial R_{11}^1}$$

STEP FUNCTION

$$\frac{\partial f_1^1}{\partial R_{11}^1} = \frac{\partial (\beta_1^1 + R_{11}^1 h_1^1 + R_{12}^1 h_2^1)}{\partial R_{11}^1} = h_1^1$$

$$\Rightarrow R_{11}^2 \frac{\partial h_1^2}{\partial R_{11}^1} = R_{11}^2 \cdot step(f_1^1) \cdot h_1^1$$

$$\frac{\partial L}{\partial R_{11}^1} = (f^2 - y) \cdot R_{11}^2 \cdot step(f_1^1) \cdot h_1 \quad \text{ALL KNOWN ELEMENTS}$$

I can reuse the result of $\frac{\partial L}{\partial R_{11}^1}$ w.r.t. $\frac{\partial L}{\partial R_{12}^1}$:

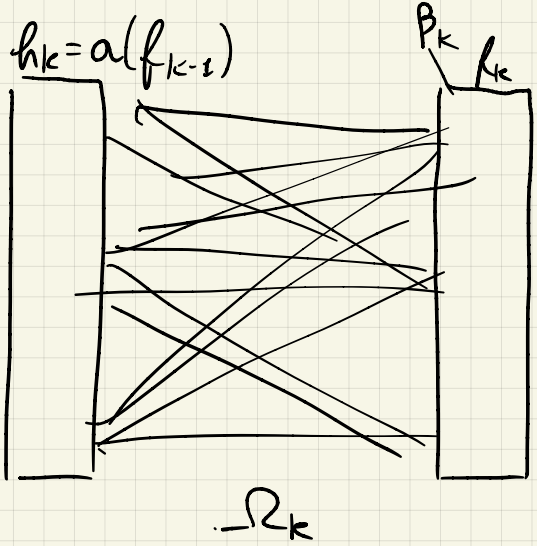
$$\frac{\partial L}{\partial R_{12}^1} = \frac{\partial L}{\partial f^2} \cdot \frac{\partial f^2}{\partial R_{12}^1} = (f^2 - y) \cdot R_{12}^2 \cdot step(f_1^1) \cdot h_2^1$$

So, BACKPROPAGATION just computes backward at every iteration of gradient descent (in computers, GPU is used because of the possibility to parallelize the computation)

PARAMETER INITIALIZATION

We start by initializing randomly. One very simple way is by a normal distribution ($\mu = 0$, σ is selected for every single matrix, $\sigma_{R_i}^2$; $i = 1, \dots, n$).

Moving to a hidden layer to another, though, can lead to relation problems wrt the importance of our weights in the hidden layers. Let's see an example:



Assuming I select some variance $\sigma^2_{\Omega_k}$ s.t. it is very small (i.e. 0,002), so our layers have a very weak relevance in our activation:

$$f_k = \beta_k + \underbrace{\Omega_k h_k}_{\text{THIS IS VERY SMALL}}$$

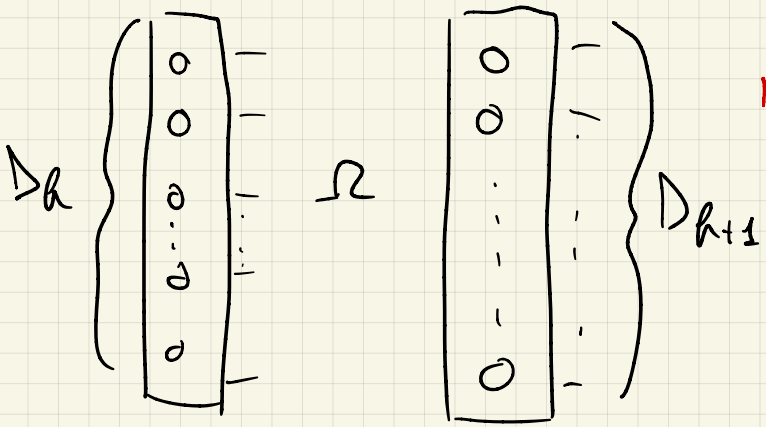
By applying the activation functions, the computation of every hidden layer will lead to tinier and tinier values. So, our convergence by applying gradient will be very slow. Also, if $\sigma^2_{\Omega_k}$ is too large, this will lead to huge values both by computing the algorithm from input to output and viceversa. This leads to overflowes and approximation errors in loss evaluation.

- TOO SMALL PARAMETERS → VANISHING GRADIENT PROBLEM
  - ↓
  - APPROXIMATION ERRORS
  - SLOW CONVERGING

- TOO LARGE PARAMETERS → EXPLODING GRADIENT PROBLEM
  ↓
  - REPRESENTATION ERRORS
  - UNSTABLE LEARNING *

* UNSTABLE LEARNING leads to a not smooth learning of the curve (this could also lead to divergence in extreme cases)

So, to still initialize the matrixes randomly avoiding both vanishing and exploding gradient problems, it is possible to choose a fixed variance:



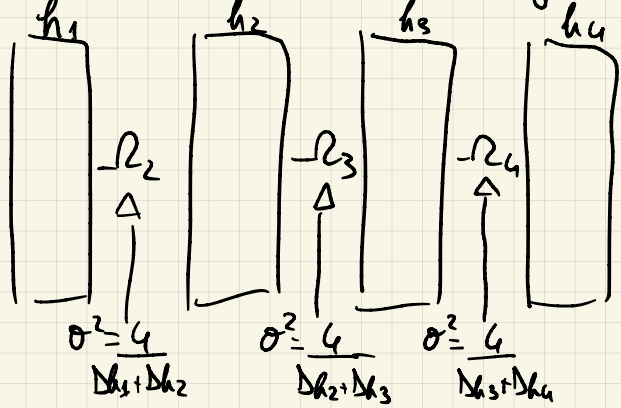
EMPIRICAL RULE

$$\sigma^2_{\Omega} = \frac{4}{D_h + D_{h+1}}$$

VARIANCE INITIALIZATION

$\sigma^2_{\Omega} = \frac{2}{D_h}$ FOR 2 LAYERS WITH SAME DIMENSION

Applied to more than 2 hidden layers:



EVERY LIBRARY HAS ITS OWN INITIALIZERS

REGULARIZATION

As for linear regression problem, a regularizer is applicable also to NN (both L_1-lasso and L_2-RIDGE are applicable).

$$J(\vartheta) = E(\vartheta) + \lambda R(\vartheta)$$

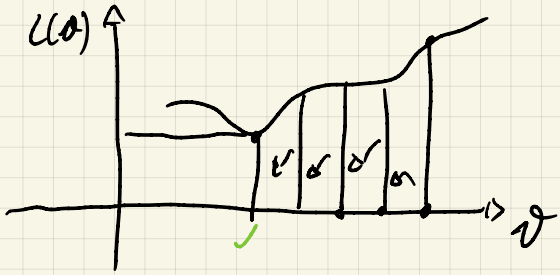
HYPERPARAMETER

$$|\vartheta| \quad (L_1)$$

$$|\vartheta|^2 \quad (L_2)$$

Also in this case, regularization is used to mitigate overfitting (in the network). We now see 2 approaches:

- EARLY STOPPING



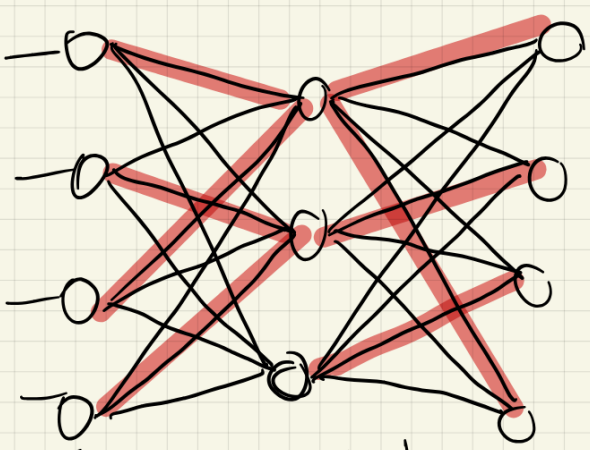
LOSS FUNCTION REDUCTION BY STOPPING THE TRAINING BEFORE REACHING MINIMUM

To understand the optimal stop time, validation could be used. The moment to stop is given when validation loss function grows up (it is exactly the moment when training loss is reaching the minimum, so it is going to overfit).

- DROPOUT

The idea is to fix a probability p of removing a link (setting its parameter to 0). At every iteration, some links

are going to drop based on p. Then, at the end of the iteration, the links drop in again and so on until a stopping criteria.



Similar to stochastic gradient descent, it is a way to avoid that the network learns "too much" from the training data.

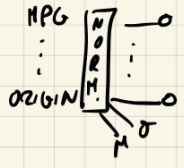
Here regularizers can be combined together.

TENSORFLOW LIBRARY TUTORIAL

TENSORFLOW AND PYTORCH ARE 2 LIBRARIES BASED ON PYTHON USED TO DEFINE A NEURAL NETWORK. BY KNOWING 1 OF THEM, LEARNING THE OTHER ONE IS VERY SIMPLE.

- NORMALIZATION LAYER

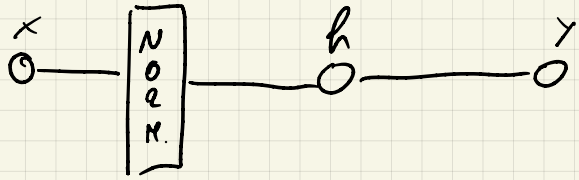
To apply normalization, a layer (not trained) is formed to give as input the normalized train features



$$z = \frac{x - \mu}{\sigma}$$

- LINEAR REGRESSION

To build the NN, start by linear regression with a single variable as input, "Horsepower".

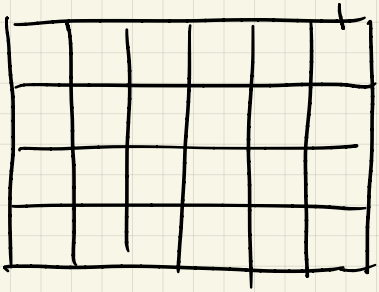


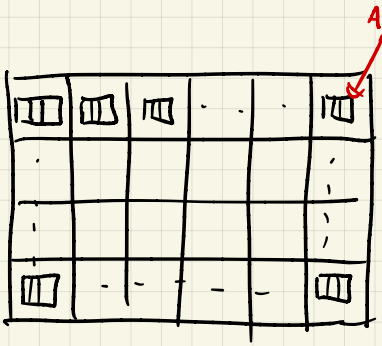
x: "Horsepower" ; h: linear transformer ; y: output

TENSORS

Tensors are an object that generalizes arrays, vectors and matrices to higher dimensions.

EXAMPLE





3D TENSOR

Tensor combine arrays and matrices by creating a structure with higher dimension to abstract objects like images, frames, ecc.